Plugins

Quick Start

A plugin is just a **small code module** that extends OpenClaw with additional functionality (commands, tools, and Gateway RPC). Most of the time, you use plugins when you want a feature that is not yet built into core OpenClaw (or you want to keep optional features out of the main installation). Quick path:

1. Check what is loaded:

```
openclaw plugins list
```

1. Install an official plugin (for example: Voice Call):

```
openclaw plugins install @openclaw/voice-call
```

1. Restart the Gateway, then configure it under `plugins.entries.<id>.config`.

Available Plugins (Official)

- Starting from 2026.1.15, Microsoft Teams is provided only as a plugin; if you use Teams, install `@openclaw/msteams`.
- Memory (Core) — the bundled memory search plugin (enabled by default via `plugins.slots.memory`)
- Memory (LanceDB) — the bundled long-term memory plugin (automatic recall/capture; set `plugins.slots.memory = "memory-lancedb"`)
- Voice Call — `@openclaw/voice-call`
- Zalo Personal — `@openclaw/zalouser`
- Matrix — `@openclaw/matrix`
- Nostr — `@openclaw/nostr`
- Zalo — `@openclaw/zalo`
- Microsoft Teams — `@openclaw/msteams`
- Google Antigravity OAuth (provider authentication) — bundled as `google-antigravity-auth` (disabled by default)
- Gemini CLI OAuth (provider authentication) — bundled as `google-gemini-cli-auth` (disabled by default)
- Qwen OAuth (provider authentication) — bundled as `qwen-portal-auth` (disabled by default)
- Copilot Proxy (provider authentication) — a local VS Code Copilot Proxy bridge; different from the built-in `github-copilot` device login (bundled, disabled by default)

OpenClaw plugins are **TypeScript modules** loaded at runtime via jiti. **Configuration validation does not execute plugin code**; it uses the plugin manifest and JSON Schema. See Plugin Manifest. A plugin can register:

- Gateway RPC methods
- Gateway HTTP handlers
- Agent tools
- CLI commands
- Background services
- Optional configuration validation
- **Skills** (by listing the `skills` directory in the plugin manifest)
- **Auto-reply commands** (executed without invoking the AI agent)

Plugins run **in the same process** as the Gateway, so treat them as trusted code. Tool authoring guide: Plugin Agent Tools.

Runtime Helpers

Plugins can access selected core helper tools via `api.runtime`. For telephony TTS:

```
const result = await api.runtime.tts.textToSpeechTelephony({
  text: "Hello from OpenClaw",
  cfg: api.config,
});
```

Notes:

- Uses the core `messages.tts` configuration (OpenAI or ElevenLabs).
- Returns a PCM audio buffer + sample rate. The plugin must resample/encode for the provider.
- Edge TTS does not support telephony.

Discovery and Priority

OpenClaw scans in this order:

1. Configuration paths

- `plugins.load.paths` (files or directories)

1. Workspace extensions

- `<workspace>/.openclaw/extensions/*.ts`
- `<workspace>/.openclaw/extensions/*/index.ts`

1. Global extensions

- `~/.openclaw/extensions/*.ts`
- `~/.openclaw/extensions/*/index.ts`

1. Bundled extensions (shipped with OpenClaw, **disabled by default**)

- `<openclaw>/extensions/*`

Bundled plugins must be explicitly enabled via `plugins.entries.<id>.enabled` or `openclaw plugins enable <id>`. Installed plugins are enabled by default, but can be disabled in the same way. Each plugin must contain an `openclaw.plugin.json` file in its root directory. If a path points to a file, the plugin root directory is the directory of the file, and it must contain the manifest. If multiple plugins resolve to the same id, the first match in the order above wins, and lower-priority copies are ignored.

Package Collections

A plugin directory can contain a `package.json` with `openclaw.extensions`:

```
{
  "name": "my-pack",
  "openclaw": {
    "extensions": ["./src/safety.ts", "./src/tools.ts"]
  }
}
```

Each entry becomes a plugin. If the package lists multiple extensions, the plugin id becomes `name/<fileBase>`. If your plugin imports npm dependencies, install them in that directory so that `node_modules` is available (`npm install` / `pnpm install`).

Channel Catalog Metadata

Channel plugins can broadcast onboarding metadata via `openclaw.channel` and installation hints via `openclaw.install`. This keeps the core catalog data-free. Example:

```
{
  "name": "@openclaw/nextcloud-talk",
  "openclaw": {
    "extensions": ["./index.ts"],
    "channel": {
      "id": "nextcloud-talk",
      "label": "Nextcloud Talk",
      "selectionLabel": "Nextcloud Talk (self-hosted)",
      "docsPath": "/channels/nextcloud-talk",
      "docsLabel": "nextcloud-talk",
      "blurb": "Self-hosted chat via Nextcloud Talk webhook bots.",
      "order": 65,
      "aliases": ["nc-talk", "nc"]
    },
    "install": {
      "npmSpec": "@openclaw/nextcloud-talk",
      "localPath": "extensions/nextcloud-talk",
      "defaultChoice": "npm"
    }
  }
}
```

OpenClaw can also merge **external channel catalogs** (for example, MPM registry exports). Place the JSON file in one of the following locations:

- `~/.openclaw/mpm/plugins.json`

- `~/.openclaw/mpm/catalog.json`
- `~/.openclaw/plugins/catalog.json`

Or point `OPENCLAW_PLUGIN_CATALOG_PATHS` (or `OPENCLAW_MPM_CATALOG_PATHS`) to one or more JSON files (separated by commas/semicolons/`PATH`). Each file should contain `{ "entries": [ { "name": "@scope/pkg", "openclaw": { "channel": {...}, "install": {...} } } ] }`.

Plugin ID

Default plugin id:

- Package collection: the `name` of `package.json`
- Standalone file: the file base name (`~/.../voice-call.ts` → `voice-call`)

If the plugin exports an `id`, OpenClaw uses it, but issues a warning when it does not match the configured id.

Configuration

```
{
  plugins: {
    enabled: true,
    allow: ["voice-call"],
    deny: ["untrusted-plugin"],
    load: { paths: ["~/Projects/oss/voice-call-extension"] },
    entries: {
      "voice-call": { enabled: true, config: { provider: "twilio" } },
    },
  },
}
```

Fields:

- `enabled`: master switch (default: true)
- `allow`: allowlist (optional)
- `deny`: denylist (optional; deny takes precedence)
- `load.paths`: additional plugin files/directories
- `entries.<id>`: per-plugin switch + configuration

Configuration changes **require restarting the Gateway**. Validation rules (strict):

- Unknown plugin ids in `entries`, `allow`, `deny`, or `slots` are an **error**.
- Unknown `channels.<id>` keys are an **error**, unless the plugin manifest declares the channel id.
- Plugin configuration is validated using the JSON Schema embedded in `openclaw.plugin.json` (`configSchema`).
- If a plugin is disabled, its configuration is retained and a **warning** is issued.

Plugin Slots (Exclusive Categories)

Some plugin categories are **exclusive** (only one active at a time). Use `plugins.slots` to select which plugin owns the slot:

```
{
  plugins: {
    slots: {
      memory: "memory-core", // or "none" to disable memory plugins
    },
  },
}
```

If multiple plugins declare `kind: "memory"`, only the selected one is loaded. The others are disabled with diagnostic information.

Control Interface (Schema + Labels)

The control interface uses `config.schema` (JSON Schema + `uiHints`) to render better forms. At runtime, OpenClaw augments `uiHints` based on the discovered plugins:

- Adds per-plugin labels for `plugins.entries.<id>` / `.enabled` / `.config`
- Merges optional plugin-provided configuration field hints at: `plugins.entries.<id>.config.<field>`

If you want plugin configuration fields to display good labels/placeholders (and to mark keys as sensitive), provide `uiHints` and JSON Schema in the plugin manifest. Example:

```
{
  "id": "my-plugin",
  "configSchema": {
    "type": "object",
    "additionalProperties": false,
    "properties": {
      "apiKey": { "type": "string" },
      "region": { "type": "string" }
    }
  },
  "uiHints": {
    "apiKey": { "label": "API Key", "sensitive": true },
    "region": { "label": "Region", "placeholder": "us-east-1" }
  }
}
```

CLI

```
openclaw plugins list
openclaw plugins info <id>
openclaw plugins install <path>                 # copy a local file/dir into
~/.openclaw/extensions/<id>
openclaw plugins install ./extensions/voice-call # relative path ok
openclaw plugins install ./plugin.tgz           # install from a local tarball
openclaw plugins install ./plugin.zip           # install from a local zip
openclaw plugins install -l ./extensions/voice-call # link (no copy) for dev
openclaw plugins install @openclaw/voice-call # install from npm
openclaw plugins update <id>
openclaw plugins update --all
openclaw plugins enable <id>
openclaw plugins disable <id>
openclaw plugins doctor
```

`plugins update` applies only to npm installations tracked under `plugins.installs`. Plugins can also register their own top-level commands (for example: `openclaw voicecall`).

Plugin API (Overview)

A plugin exports one of the following:

- A function: `(api) => { ... }`
- An object: `{ id, name, configSchema, register(api) { ... } }`

Plugin Hooks

Plugins can ship hooks and register them at runtime. This lets a plugin bundle event-driven automation without installing a separate hook package.

Example

```
import { registerPluginHooksFromDir } from "openclaw/plugin-sdk";

export default function register(api) {
  registerPluginHooksFromDir(api, "./hooks");
}
```

Notes:

- Hook directories follow the normal hook structure (`HOOK.md` + `handler.ts`).
- Hook eligibility rules still apply (OS/binary/environment/configuration requirements).
- Hooks managed by a plugin appear as `plugin:<id>` in `openclaw hooks list`.
- You cannot enable/disable plugin-managed hooks via `openclaw hooks`; instead, enable/disable the plugin.

Provider Plugins (Model Authentication)

Plugins can register **model provider authentication** flows so that users can run OAuth or API key setup inside OpenClaw (no external scripts required). Register a provider via `api.registerProvider(...)`. Each provider exposes one or more authentication methods (OAuth, API key, device code, etc.). These methods drive:

- `openclaw models auth login --provider <id> [--method <id>]`

Example:

```
api.registerProvider({
  id: "acme",
  label: "AcmeAI",
  auth: [
    {
      id: "oauth",
      label: "OAuth",
      kind: "oauth",
      run: async (ctx) => {
        // Run OAuth flow and return auth profiles.
        return {
          profiles: [
            {
              profileId: "acme:default",
              credential: {
                type: "oauth",
                provider: "acme",
                access: "...",
                refresh: "...",
                expires: Date.now() + 3600 * 1000,
              },
            },
          ],
          defaultModel: "acme/opus-1",
        };
      },
    },
  ],
});
```

Notes:

- `run` receives a `ProviderAuthContext` with the `prompter`, `runtime`, `openUrl`, and `oauth.createVpsAwareHandlers` helper tools.
- Return a `configPatch` when a default model or provider configuration needs to be added.
- Return `defaultModel` so that `--set-default` can update the agent defaults.

Registering Message Channels

Plugins can register **channel plugins** that behave like built-in channels (WhatsApp, Telegram, etc.). Channel configuration lives under `channels.<id>` and is validated by your channel plugin code.

```
const myChannel = {
  id: "acmechat",
  meta: {
    id: "acmechat",
    label: "AcmeChat",
    selectionLabel: "AcmeChat (API)",
    docsPath: "/channels/acmechat",
    blurb: "demo channel plugin.",
    aliases: ["acme"],
  },
  capabilities: { chatTypes: ["direct"] },
  config: {
    listAccountIds: (cfg) => Object.keys(cfg.channels?.acmechat?.accounts ?? {}),
    resolveAccount: (cfg, accountId) =>
      cfg.channels?.acmechat?.accounts?.[accountId ?? "default"] ?? {
        accountId,
      },
  },
  outbound: {
    deliveryMode: "direct",
    sendText: async () => ({ ok: true }),
  },
};

export default function (api) {
  api.registerChannel({ plugin: myChannel });
}
```

Notes:

- Place the configuration under `channels.<id>` (not `plugins.entries`).
- `meta.label` is used for the label in CLI/UI lists.
- `meta.aliases` adds alternate ids used for normalization and CLI input.
- `meta.preferOver` lists channel ids to skip auto-enabling when both are configured.
- `meta.detailLabel` and `meta.systemImage` let the UI display richer channel labels/icons.

Writing a New Message Channel (Step-by-Step Guide)

Use this approach when you want a **new chat interface** ("message channel") rather than a model provider. Model provider documentation is located under `/providers/*`.

1. Choose the id + configuration structure

- All channel configuration lives under `channels.<id>`.
- For multi-account setups, prefer `channels.<id>.accounts.<accountId>`.

1. Define the channel metadata

- `meta.label`, `meta.selectionLabel`, `meta.docsPath`, `meta.blurb` control the CLI/UI list.
- `meta.docsPath` should point to a documentation page like `/channels/<id>`.
- `meta.preferOver` lets the plugin replace another channel (auto-enabling prefers it).
- `meta.detailLabel` and `meta.systemImage` are used by the UI for detailed text/icons.

1. Implement the required adapters

- `config.listAccountIds` + `config.resolveAccount`
- `capabilities` (chat types, media, threading, etc.)
- `outbound.deliveryMode` + `outbound.sendText` (for basic sending)

1. Add optional adapters as needed

- `setup` (wizard), `security` (direct message policy), `status` (health/diagnostics)
- `gateway` (start/stop/login), `mentions`, `threading`, `streaming`
- `actions` (message actions), `commands` (native command behavior)

1. Register the channel in the plugin

- `api.registerChannel({ plugin })`

Minimal configuration example:

```
{
  channels: {
    acmechat: {
      accounts: {
        default: { token: "ACME_TOKEN", enabled: true },
      },
    },
  },
}
```

Minimal channel plugin (outbound only):

```
const plugin = {
  id: "acmechat",
  meta: {
    id: "acmechat",
    label: "AcmeChat",
    selectionLabel: "AcmeChat (API)",
    docsPath: "/channels/acmechat",
    blurb: "AcmeChat messaging channel.",
    aliases: ["acme"],
  },
  capabilities: { chatTypes: ["direct"] },
  config: {
    listAccountIds: (cfg) => Object.keys(cfg.channels?.acmechat?.accounts ?? {}),
    resolveAccount: (cfg, accountId) =>
      cfg.channels?.acmechat?.accounts?.[accountId ?? "default"] ?? {
        accountId,
      },
  },
```

```
  outbound: {
    deliveryMode: "direct",
    sendText: async ({ text }) => {
      // deliver `text` to your channel here
      return { ok: true };
    },
  },
};

export default function (api) {
  api.registerChannel({ plugin });
}
```

Load the plugin (extension directory or `plugins.load.paths`), restart the Gateway, then configure `channels.<id>` in the configuration.

Agent Tools

See the dedicated guide: Plugin Agent Tools.

Registering Gateway RPC Methods

```
export default function (api) {
  api.registerGatewayMethod("myplugin.status", ({ respond }) => {
    respond(true, { ok: true });
  });
}
```

Registering CLI Commands

```
export default function (api) {
  api.registerCli(
    ({ program }) => {
      program.command("mycmd").action(() => {
        console.log("Hello");
      });
    },
    { commands: ["mycmd"] },
  );
}
```

Registering Auto-Reply Commands

Plugins can register custom slash commands that execute **without invoking the AI agent**. This is useful for toggle commands, status checks, or quick actions that do not need LLM processing.

```
export default function (api) {
  api.registerCommand({
    name: "mystatus",
    description: "Show plugin status",
    handler: (ctx) => ({
      text: `Plugin is running! Channel: ${ctx.channel}`,
    }),
  });
}
```

Command handler context:

- `senderId`: the sender's ID (if available)
- `channel`: the channel that sent the command
- `isAuthorizedSender`: whether the sender is an authorized user
- `args`: the arguments passed after the command (if `acceptsArgs: true`)
- `commandBody`: the full command text
- `config`: the current OpenClaw configuration

Command options:

- `name`: the command name (without the leading `/`)
- `description`: the help text displayed in the command list
- `acceptsArgs`: whether the command accepts arguments (default: false). If false and arguments are provided, the command does not match and the message is passed to other handlers
- `requireAuth`: whether an authorized sender is required (default: true)
- `handler`: a function that returns `{ text: string }` (can be asynchronous)

Example with authorization and arguments:

```
api.registerCommand({
  name: "setmode",
  description: "Set plugin mode",
  acceptsArgs: true,
  requireAuth: true,
  handler: async (ctx) => {
    const mode = ctx.args?.trim() || "default";
    await saveMode(mode);
    return { text: `Mode set to: ${mode}` };
  },
});
```

Notes:

- Plugin commands are processed **before** built-in commands and the AI agent
- Commands are registered globally and apply to all channels
- Command names are case-insensitive (`/MyStatus` matches `/mystatus`)
- Command names must start with a letter and may contain only letters, digits, hyphens, and underscores

- Reserved command names (such as `help`, `status`, `reset`, etc.) cannot be overridden by plugins
- Duplicate command registration across plugins fails with a diagnostic error

Registering Background Services

```
export default function (api) {
  api.registerService({
    id: "my-service",
    start: () => api.logger.info("ready"),
    stop: () => api.logger.info("bye"),
  });
}
```

Naming Conventions

- Gateway methods: `pluginId.action` (for example: `voicecall.status`)
- Tools: `snake_case` (for example: `voice_call`)
- CLI commands: kebab or camel, but avoid conflicts with core commands

Skills

A plugin can ship Skills in the repository (`skills/<name>/SKILL.md`). Enable it with `plugins.entries.<id>.enabled` (or another configuration gate), and make sure it exists in your workspace/managed Skills location.

Distribution (npm)

Recommended packaging approach:

- Main package: `openclaw` (this repository)
- Plugins: independent npm packages under `@openclaw/*` (for example: `@openclaw/voice-call`)

Publishing contract:

- The plugin `package.json` must contain `openclaw.extensions` with one or more entry files.
- Entry files can be `.js` or `.ts` (jiti loads TS at runtime).
- `openclaw plugins install <npm-spec>` uses `npm pack`, extracts to `~/.openclaw/extensions/<id>/`, and enables it in the configuration.
- Configuration key stability: scoped packages are normalized to an **unscoped** id for `plugins.entries.*`.

Example Plugin: Voice Call

This repository contains a voice call plugin (Twilio or a log fallback):

- Source: `extensions/voice-call`
- Skills: `skills/voice-call`
- CLI: `openclaw voicecall start|status`

- Tool: `voice_call`
- RPC: `voicecall.start`, `voicecall.status`
- Configuration (twilio): `provider: "twilio"` + `twilio.accountSid/authToken/from` (optional `statusCallbackUrl`, `twimlUrl`)
- Configuration (dev): `provider: "log"` (no network)

See Voice Call and `extensions/voice-call/README.md` for setup and usage.

Security Considerations

Plugins run in the same process as the Gateway. Treat them as trusted code:

- Install only plugins you trust.
- Prefer the `plugins.allow` allowlist.
- Restart the Gateway after changes.

Testing Plugins

Plugins can (and should) ship with tests:

- In-repository plugins can keep Vitest tests under `src/**` (for example: `src/plugins/voice-call.plugin.test.ts`).
- Separately published plugins should run their own CI (lint/build/test) and verify that `openclaw.extensions` points to the built entry point (`dist/index.js`).

For more about plugins, refer to: Plugins - OpenClaw